Bubble sort works by repeatedly comparing adjacent elements and swapping them if they are out of order. Cormen *et al.* (2022, p. 46) describe it as a popular but inefficient sorting algorithm. Each pass through the list moves the largest unsorted value to its correct position, with the process repeating until no swaps are required. This straightforward approach makes the algorithm easy to understand and implement, which explains its common use in educational contexts.

This simplicity carries a significant performance penalty. Bubble sort exhibits $O(n^2)$ time complexity, meaning that doubling the input size quadruples the processing time. Quicksort operates fundamentally differently, using a divide-and-conquer approach that partitions the data around a pivot element and recursively sorts the resulting subsets (Cormen *et al.*, 2022, p. 182). This strategy yields $O(n \log n)$ expected performance. Both algorithms were tested against randomly generated datasets of 100, 1,000, 10,000, and 100,000 elements. At 100 elements, both algorithms completed almost instantaneously. However, as the input size increased, the performance gap widened dramatically. Bubble sort required over 600 seconds to process 100,000 elements, whereas quicksort completed the identical task in under one second.

These findings carry practical implications for software engineering. In test automation, execution time directly affects whether teams actually run their test suites. When tests take too long, developers face pressure to skip them, undermining quality assurance entirely. A test suite built on inefficient algorithms may function correctly yet fail in practice because no one executes it. This pattern extends to continuous integration pipelines, monitoring systems, and data processing workflows. Bubble sort remains suitable for small, bounded datasets where simplicity matters more than speed. Production systems handling larger or variable data volumes require efficient algorithms, such as quicksort, to maintain acceptable performance at scale.

Reference:

Cormen, T.H. *et al.* (2022) *Introduction to Algorithms*. 4th edn. Cambridge, MA: MIT Press.